

Data Preparation for Machine Learning

Compilation: Yogesh Haribhau Kulkarni

Data Preparation

Introduction to Pandas

Pandas

pandas introduces two new data structures to Python - **Series** and **DataFrame**, both of which are built on top of NumPy. Throughout this session, lines starting with `>>>` (or ... for a continued line) are what you type at the Python prompt; everything else is the output pandas prints back.

```
>>> import pandas as pd
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> pd.set_option('display.max_columns', 50)
```

Series

Series is a one-dimensional labeled array capable of holding any data type (integers, strings, floating point numbers, Python objects, etc.). The axis labels are collectively referred to as the index. The basic method to create a Series is to call:

Here, data can be many different things:

- a Python dict
- an ndarray
- a scalar value (like 5)

```
>>> s = pd.Series(data, index=index)
```

Creating a Series from a List

- A Series is a one-dimensional object similar to an array, list, or column in a table.
- It will assign a labeled index to each item in the Series.
- By default, each item will receive an index label from 0 to N, where N is the length of the Series minus one.

```
>>> # create a Series with an arbitrary list
>>> s = pd.Series([7, 'Heisenberg', 3.14,
                  -1789710578, 'Happy Eating!'])
>>> s
```

Series

Output from Previous Slide

```
0          7
1    Heisenberg
2         3.14
3   -1789710578
4    Happy Eating!
dtype: object
```

Series with a Custom Index

Alternatively, you can specify an index to use when creating the Series.

```
>>> s = pd.Series([7, 'Heisenberg', 3.14,
                  -1789710578,
                  'Happy Eating!'],
                  index=['A', 'Z', 'C', 'Y', 'E'])
>>> s

A          7
Z    Heisenberg
C         3.14
Y   -1789710578
E    Happy Eating!
dtype: object
```

Series

The Series constructor can convert a dictionary as well, using the keys of the dictionary as its index.

```
>>> d = {'Chicago': 1000, 'New York': 1300,
        'Portland': 900, 'San Francisco':
        1100, 'Austin': 450, 'Boston': None}
>>> cities = pd.Series(d)
>>> cities

Austin          450
Boston          NaN
Chicago        1000
New York       1300
Portland         900
San Francisco   1100
dtype: float64
```

Series

You can use the index to select specific items from the Series

```
>>> cities['Chicago']
1000.0
```

Series

```
>>> cities[['Chicago', 'Portland', 'San
          Francisco']]
Chicago          1000
Portland          900
San Francisco    1100
dtype: float64
```

Series

You can use *boolean indexing* for selection.

```
>>> cities[cities < 1000]
Austin          450
Portland         900
dtype: float64
```

Boolean Indexing, Explained

That last one might be a little strange, so let's make it more clear - `cities < 1000` returns a Series of `True/False` values, which we then pass to our Series `cities`, returning the corresponding `True` items.

Boolean Indexing: Selecting Values

```
>>> less_than_1000 = cities < 1000
>>> print(less_than_1000)
>>> print('\n')
>>> print(cities[less_than_1000])
Austin          True
Boston          False
Chicago          False
New York         False
Portland          True
San Francisco    False
dtype: bool

Austin          450
Portland         900
dtype: float64
```

Modifying Series Values by Index

You can also change the values in a Series on the fly.

```
>>> # changing based on the index
>>> print('Old value:', cities['Chicago'])
>>> cities['Chicago'] = 1400
>>> print('New value:', cities['Chicago'])
Old value: 1000.0
New value: 1400.0
```

Modifying Values via Boolean Logic

Changing values using boolean logic

```
>>> print(cities[cities < 1000])
>>> print('\n')
>>> cities[cities < 1000] = 750
>>> print(cities[cities < 1000])
Austin      450
Portland    900
dtype: float64

Austin      750
Portland    750
dtype: float64
```

Working with Series

What if you aren't sure whether an item is in the Series? You can check using idiomatic Python.

```
>>> print('Seattle' in cities)
>>> print('San Francisco' in cities)
False
True
```

Series Arithmetic with Scalars

Mathematical operations can be done using scalars and functions.

```
>>> # divide city values by 3
>>> cities / 3
Austin      250.000000
Boston      NaN
Chicago     466.666667
New York    433.333333
Portland    250.000000
San Francisco 366.666667
dtype: float64
```

Series Arithmetic with NumPy Functions

```
>>> # square city values
>>> np.square(cities)
Austin      562500
Boston      NaN
Chicago     1960000
New York    1690000
Portland    562500
San Francisco 1210000
dtype: float64
```

Adding Two Series Together

You can add two Series together, which returns a union of the two Series with the addition occurring on the shared index values. Values on either Series that did not have a shared index will produce a NULL/NaN (not a number).

```
>>> print(cities[['Chicago', 'New York',
                  'Portland']])
>>> print('\n')
>>> print(cities[['Austin', 'New York']])
>>> print('\n')
>>> print(cities[['Chicago', 'New York',
                  'Portland']] + cities[['Austin', 'New York']])
```

Adding Two Series: Output

```
Chicago     1400
New York    1300
Portland     750
dtype: float64

Austin      750
New York    1300
dtype: float64

Austin      NaN
Chicago     NaN
New York    2600
Portland    NaN
dtype: float64
```

Working with Series

NULL Checking

- Notice that because Austin, Chicago, and Portland were not found in both Series, they were returned with NULL/NaN values.
- NULL checking can be performed with `isnull()` and `notnull()`.

Intuition: Index Alignment

Intuition

Adding two Series is like matching up name tags at two different parties and adding values only where the *same* name appears at both. Anyone who only showed up to one party (an index value present in just one Series) gets a NaN back: there's nothing on the other side to add them to.

Checking for Non-Null Values

Return a boolean series indicating which values aren't NULL

```
>>> cities.notnull()

Austin      True
Boston      False
Chicago     True
New York    True
Portland    True
San Francisco True
dtype: bool
```

Checking for Null Values

Using boolean logic to grab the NULL cities

```
>>> print(cities.isnull())
>>> print('\n')
>>> print(cities[cities.isnull()])
Austin      False
Boston      True
Chicago     False
New York    False
Portland    False
San Francisco False
dtype: bool

Boston      NaN
dtype: float64
```

Quick Check: Series

Think About It

`cities['Chicago']` returns a single value, but `cities[['Chicago', 'Portland']]` returns a Series. What's the difference in the syntax, and why does it matter?

Quick Check: Series

Answer

A single string key does scalar/label lookup and returns *one* value. A *list* of keys is fancy indexing and always returns a Series, even with just one element: the double brackets are

doing two things at once: the outer `[]` is the indexing operation, the inner `[]` is a literal Python list being passed as the key.

Dataframes in Pandas

Creating Dataframes

Creating a DataFrame by passing a numpy array, with a date-time index and labeled columns:

```
>>> dates = pd.date_range('20130101', periods=6)
>>> dates
DatetimeIndex(['2013-01-01', '2013-01-02',
               '2013-01-03', '2013-01-04',
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')

>>> df_demo = pd.DataFrame(np.random.randn(6, 4),
                           index=dates, columns=list('ABCD'))
>>> df_demo
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401
2013-01-06	-0.673690	0.113648	-1.478427	0.524988

Creating Dataframes

Creating a DataFrame by passing a dict of objects that can be converted to series-like.

```
>>> df2 = pd.DataFrame({'A': 1.,
...                     'B':
...     pd.Timestamp('20130102'),
...                     'C': pd.Series(1,
...     index=list(range(4)), dtype='float32'),
...                     'D': np.array([3] * 4,
...     dtype='int32'),
...                     'E':
...     pd.Categorical(["test", "train", "test",
...     "train"]),
...                     'F': 'foo'})
>>> df2
```

	A	B	C	D	E	F
0	1.0	2013-01-02	1.0	3	test	foo
1	1.0	2013-01-02	1.0	3	train	foo
2	1.0	2013-01-02	1.0	3	test	foo
3	1.0	2013-01-02	1.0	3	train	foo

Reading Data from Files

Every DataFrame so far has been typed in by hand. In practice, almost all data starts life in a file: `pd.read_csv()` and `pd.read_excel()` load it straight into a DataFrame, inferring column names and types automatically. `machine_logs.csv` is provided upfront (no need to type it): copy it into your working folder before class.

```
>>> df = pd.read_csv('machine_logs.csv')
>>> df.head(3)
```

	machine_id	shift	temperature	vibration	operator_notes
0	M1	Morning	72.5	0.021	ok
1	M2	Morning	75.1	0.034	check bearing
2	M1	Evening	78.3	0.041	ok

Quick Inspection

Before doing anything else with a newly loaded DataFrame, check its shape, column types, and non-null counts.

```
>>> df.shape
(6, 5)

>>> df.dtypes
machine_id      object
shift           object
temperature     float64
vibration       float64
operator_notes  object
dtype: object
```

Quick Inspection: Full Structure

`df.info()` combines shape, dtypes, and non-null counts into one summary.

```
>>> df.info()
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 6 entries, 0 to 5
Data columns (total 5 columns):
#   Column          Non-Null Count  Dtype
---  -
0   machine_id      6 non-null     object
1   shift           6 non-null     object
2   temperature     6 non-null     float64
3   vibration       6 non-null     float64
4   operator_notes  6 non-null     object
dtypes: float64(2), object(3)
```

Viewing Data

See the top & bottom rows of the frame

```
>>> df_demo.head()
```

	A	B	C	D
--	---	---	---	---

```
2013-01-01  0.469112 -0.282863 -1.509059 -1.135632
2013-01-02  1.212112 -0.173215  0.119209 -1.044236
2013-01-03 -0.861849 -2.104569 -0.494929  1.071804
2013-01-04  0.721555 -0.706771 -1.039575  0.271860
2013-01-05 -0.424972  0.567020  0.276232 -1.087401

>>> df_demo.tail(3)
```

	A	B	C	D
2013-01-04	0.721555	-0.706771	-1.039575	0.271860
2013-01-05	-0.424972	0.567020	0.276232	-1.087401
2013-01-06	-0.673690	0.113648	-1.478427	0.524988

Viewing Data

Display the index, columns, and the underlying numpy data

```
>>> df_demo.index
DatetimeIndex(['2013-01-01', '2013-01-02',
               '2013-01-03', '2013-01-04',
               '2013-01-05', '2013-01-06'],
              dtype='datetime64[ns]', freq='D')

>>> df_demo.columns
Index(['A', 'B', 'C', 'D'], dtype='object')

>>> df_demo.values
array([[ 0.4691, -0.2829, -1.5091, -1.1356],
       [ 1.2121, -0.1732,  0.1192, -1.0442],
       [-0.8618, -2.1046, -0.4949,  1.0718],
       [ 0.7216, -0.7068, -1.0396,  0.2719],
       [-0.425 ,  0.567 ,  0.2762, -1.0874],
       [-0.6737,  0.1136, -1.4784,  0.525 ]])
```

Viewing Data

Describe shows a quick statistic summary of your data

```
>>> df_demo.describe()
```

	A	B	C	D
count	6.000000	6.000000	6.000000	6.000000
mean	0.073711	-0.431125	-0.687758	-0.233103
std	0.843157	0.922818	0.779887	0.973118
min	-0.861849	-2.104569	-1.509059	-1.135632
25%	-0.611510	-0.600794	-1.368714	-1.076610
50%	0.022070	-0.228039	-0.767252	-0.386188
75%	0.658444	0.041933	-0.034326	0.461706
max	1.212112	0.567020	0.276232	1.071804

Viewing Data

Sorting by an axis

```
>>> df_demo.sort_index(axis=1, ascending=False)
```

	D	C	B	A
--	---	---	---	---

2013-01-01	-1.135632	-1.509059	-0.282863	0.469112
2013-01-02	-1.044236	0.119209	-0.173215	1.212112
2013-01-03	1.071804	-0.494929	-2.104569	-0.861849
2013-01-04	0.271860	-1.039575	-0.706771	0.721555
2013-01-05	-1.087401	0.276232	0.567020	-0.424972
2013-01-06	0.524988	-1.478427	0.113648	-0.673690

Viewing Data

Sorting by values

```
>>> df_demo.sort_values(by='B')
```

	A	B	C	D
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.721860
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-06	-0.673690	0.113648	-1.478427	0.524988
2013-01-05	-0.424972	0.567020	0.276232	-1.087401

Value Counts

`value_counts()` is the fastest way to see how a categorical column is distributed: here, how many readings each machine contributed.

```
>>> df['machine_id'].value_counts()
M1    2
M2    2
M3    2
Name: machine_id, dtype: int64
```

Selection of Data

Selecting a single column, which yields a Series, equivalent to `df_demo.A`

```
>>> df_demo['A']
2013-01-01    0.469112
2013-01-02    1.212112
2013-01-03   -0.861849
2013-01-04    0.721555
2013-01-05   -0.424972
2013-01-06   -0.673690
Freq: D, Name: A, dtype: float64
```

Selection of Data

Selecting via `[]`, which slices the rows.

```
>>> df_demo[0:3]
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236

2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
------------	-----------	-----------	-----------	----------

```
>>> df_demo['20130102':'20130104']
```

	A	B	C	D
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-03	-0.861849	-2.104569	-0.494929	1.071804
2013-01-04	0.721555	-0.706771	-1.039575	0.721860

Selection of Data

Selecting on a multi-axis by label

```
>>> df_demo.loc[:, ['A', 'B']]
```

	A	B
2013-01-01	0.469112	-0.282863
2013-01-02	1.212112	-0.173215
2013-01-03	-0.861849	-2.104569
2013-01-04	0.721555	-0.706771
2013-01-05	-0.424972	0.567020
2013-01-06	-0.673690	0.113648

Selection of Data

By integer slices, acting similar to numpy/python

```
>>> df_demo.iloc[3:5, 0:2]
```

	A	B
2013-01-04	0.721555	-0.706771
2013-01-05	-0.424972	0.567020

Selection of Data

By lists of integer position locations, similar to the numpy/python style

```
>>> df_demo.iloc[[1, 2, 4], [0, 2]]
```

	A	C
2013-01-02	1.212112	0.119209
2013-01-03	-0.861849	-0.494929
2013-01-05	-0.424972	0.276232

Intuition: Label vs. Position

Intuition

Label-based selection is like finding a book by its title on the shelf: you ask for “Wednesday’s row” or “column A,” and pandas looks up that name. Position-based selection is like grabbing the third book from the left, regardless of what it’s called. Both answer valid but different questions, so pandas gives you separate ways to ask each one instead of guessing your intent.

Boolean Indexing

Using a single column’s values to select data.

```
>>> df_demo[df_demo.A > 0]
```

	A	B	C	D
2013-01-01	0.469112	-0.282863	-1.509059	-1.135632
2013-01-02	1.212112	-0.173215	0.119209	-1.044236
2013-01-04	0.721555	-0.706771	-1.039575	0.721860

Setting Values

Two changes happen here that carry into every slide from this point on: column D is overwritten entirely, and a new column F is added by assigning a Series that only covers *some* of the dates: alignment (the same mechanism from the Series section) fills in the rest with NaN.

```
>>> df_demo.loc[:, 'D'] = np.array([5] *
len(df_demo))

>>> s1 = pd.Series([1, 2, 3, 4, 5, 6],
index=pd.date_range('20130102', periods=6))
>>> df_demo['F'] = s1
>>> df_demo
```

	A	B	C	D	F
2013-01-01	0.469112	-0.282863	-1.509059	5	NaN
2013-01-02	1.212112	-0.173215	0.119209	5	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5	2.0
2013-01-04	0.721555	-0.706771	-1.039575	5	3.0
2013-01-05	-0.424972	0.567020	0.276232	5	4.0
2013-01-06	-0.673690	0.113648	-1.478427	5	5.0

Missing Data

Missing values show up as NaN. Blanking out an earlier value and reindexing to a smaller set of dates creates some gaps to demonstrate on:

```
>>> df_demo.loc[dates[0], ['A', 'B']] = 0
>>> df1 = df_demo.reindex(index=dates[0:4], columns=list(df_demo.columns) +
['E'])
>>> df1.loc[dates[0]:dates[1], 'E'] = 1
>>> df1
```

	A	B	C	D	F	E
2013-01-01	0.000000	0.000000	-1.509059	5	NaN	1.0
2013-01-02	1.212112	-0.173215	0.119209	5	1.0	1.0
2013-01-03	-0.861849	-2.104569	-0.494929	5	2.0	NaN
2013-01-04	0.721555	-0.706771	-1.039575	5	3.0	NaN

Missing Data: Drop

To drop any rows that have missing data:

```
>>> df1.dropna(how='any')
```

	A	B	C	D	F	E
2013-01-02	1.212112	-0.173215	0.119209	5	1.0	1.0

Missing Data: Fill

Filling missing data:

```
>>> df1.fillna(value=5)
      A         B         C  D  F  E
2013-01-01  0.000000  0.000000 -1.509059  5  5.0  1.0
2013-01-02  1.212112 -0.173215  0.119209  5  1.0  1.0
2013-01-03 -0.861849 -2.104569 -0.494929  5  2.0  5.0
2013-01-04  0.721555 -0.706771 -1.039575  5  3.0  5.0
```

Stats

Operations in general exclude missing data. Performing a descriptive statistic:

```
>>> df_demo.mean()
A    -0.004474
B    -0.383981
C    -0.687758
D     5.000000
F     3.000000
dtype: float64
```

Stats: The Other Axis

Same operation on the other axis:

```
>>> df_demo.mean(1)
2013-01-01    0.872735
2013-01-02    1.431621
2013-01-03    0.707731
2013-01-04    1.395042
2013-01-05    1.883656
2013-01-06    1.592306
Freq: D, dtype: float64
```

Apply

Applying functions to the data

```
>>> df_demo.apply(np.cumsum)
      A         B         C  D  F
2013-01-01  0.000000  0.000000 -1.509059  5  NaN
2013-01-02  1.212112 -0.173215 -1.389850 10  1.0
2013-01-03  0.350263 -2.277784 -1.884779 15  3.0
2013-01-04  1.071818 -2.984555 -2.924354 20  6.0
2013-01-05  0.646846 -2.417535 -2.648122 25 10.0
2013-01-06 -0.026844 -2.303886 -4.126549 30 15.0

>>> df_demo.apply(lambda x: x.max() - x.min())
A    2.073961
B    2.671590
C    1.785291
D    0.000000
F    4.000000
dtype: float64
```

Renaming & Dropping Columns

Cleaning up column names and removing ones you don't need is usually the first step of any real analysis.

```
>>> df = df.rename(columns={'vibration':
                             'vibration_mm_s'})
>>> df = df.drop(columns=['operator_notes'])
>>> df.columns
Index(['machine_id', 'shift', 'temperature',
       'vibration_mm_s'], dtype='object')
```

Creating New Columns

New columns are computed from existing ones with plain vectorized arithmetic and comparisons, no loops needed.

```
>>> df['vibration_in'] = df['vibration_mm_s'] /
25.4
>>> df['high_vibration'] = df['vibration_mm_s'] >
0.025
>>> df[['machine_id', 'vibration_mm_s',
       'high_vibration']]

   machine_id  vibration_mm_s  high_vibration
0          M1           0.021             False
1          M2           0.034              True
2          M1           0.041              True
3          M2           0.028              True
4          M3           0.019             False
5          M3           0.052              True
```

Quick Check: DataFrames

Think About It

`df['A']` returns a Series (a column), but `df[0:2]` returns a DataFrame (the first two rows). Same `[]` operator, same object: why does pandas treat a string key and a slice so differently?

Quick Check: DataFrames

Answer

The plain `[]` operator on a DataFrame is intentionally overloaded: a single label looks up a *column* (matching the `df.A` shortcut), while a slice looks up *rows*, mirroring plain Python/NumPy slicing. Pandas infers your intent from the *type* of the key rather than a fixed row/column rule, which is exactly why `.loc/.iloc` exist: they remove the ambiguity by always meaning “by label” or “by position,” regardless of what kind of key you pass.

GroupBy: Split-Apply-Combine

`groupby()` splits rows into groups by a key column. Nothing is computed until you chain on an aggregate.

Intuition

Think of `groupby()` as sorting mail into pigeonholes by recipient. Nothing gets read or totalled yet: that's just the sorting step. The real work happens when a clerk (an aggregate like `.mean()` or `.sum()`) processes each pigeonhole separately and the results are stacked back into a single tray.

```
>>> df.groupby('machine_id')
<pandas.core.groupby.generic.DataFrameGroupBy
object at 0x...>
```

GroupBy with Aggregation

Chain `.mean()`, `.sum()`, or `.agg()` to actually run the per-group computation.

```
>>> df.groupby('machine_id')['temperature'].mean()
M1    75.40
M2    74.55
M3    75.50
Name: temperature, dtype: float64

>>> df.groupby('machine_id').agg({'temperature':
                                'mean', 'vibration_mm_s': 'max'})
      temperature  vibration_mm_s
machine_id
M1             75.40           0.041
M2             74.55           0.034
M3             75.50           0.052
```

Quick Check: GroupBy

Think About It

`df.groupby('machine_id')` on its own prints only an object description, not a table of numbers. Why doesn't pandas compute anything at that point?

Quick Check: GroupBy

Answer

`groupby()` only performs the *split*: it doesn't yet know what you want to *apply* to each group. Waiting until you chain on `.mean()`, `.sum()`, or `.agg()` means the same grouped object can be reused for several different aggregations without repeating the split step, and pandas never computes a result you never asked for.

Concatenating DataFrames

`pd.concat()` stacks DataFrames with the same columns on top of each other, useful when the same kind of data arrives in separate batches (e.g., one file per shift).

```
>>> morning = df[df['shift'] ==
...             'Morning'][['machine_id', 'temperature']]
>>> evening = df[df['shift'] ==
...              'Evening'][['machine_id', 'temperature']]
>>> pd.concat([morning, evening])
  machine_id  temperature
0          M1          72.5
1          M2          75.1
4          M3          69.8
2          M1          78.3
3          M2          74.0
5          M3          81.2
```

Merging DataFrames

`pd.merge()` joins two DataFrames on a shared key column: like a VLOOKUP, or a SQL JOIN.

```
>>> machine_info = pd.DataFrame({
...     'machine_id': ['M1', 'M2', 'M3'],
...     'machine_type': ['Lathe', 'Mill', 'Drill']
... })
>>> pd.merge(df, machine_info, on='machine_id')[
...     ['machine_id', 'machine_type',
...      'temperature']].head(3)
  machine_id machine_type  temperature
0          M1         Lathe          72.5
1          M2          Mill          75.1
2          M1         Lathe          78.3
```

Plotting with Pandas

`.plot()` is a thin wrapper around the `matplotlib` imported all the way back on the very first Pandas slide, no separate charting library needed for a first look at the data.

```
>>> avg_temp =
...     df.groupby('machine_id')['temperature'].mean()
>>> avg_temp.plot(kind='bar')
>>> plt.ylabel('Avg Temperature (deg C)')
>>> plt.show()
```

Putting It Together: Machine Health Check

The same five-step pattern (read, inspect, clean, transform, summarize) shows up in almost every dataset you'll touch in this course.

```
>>> df = pd.read_csv('machine_logs.csv')
>>> df = df.rename(columns={'vibration':
...                       'vibration_mm_s'})
>>> df = df.drop(columns=['operator_notes'])
>>> df['high_vibration'] = df['vibration_mm_s'] >
... 0.025

>>> summary = df.groupby('machine_id').agg(
...     avg_temp=('temperature', 'mean'),
...     flagged_readings=('high_vibration', 'sum')
... )
>>> summary
           avg_temp  flagged_readings
machine_id
M1              75.40                 1
M2              74.55                 2
M3              75.50                 1
```

Putting It Together: Visualizing the Summary

One line turns the summary table into a chart flagging which machine needs a closer look.

```
>>> summary['avg_temp'].plot(kind='bar',
...                          color='steelblue')
>>> plt.title('Average Temperature by Machine')
>>> plt.ylabel('deg C')
>>> plt.show()
```

Try It Yourself

Using the same `machine_logs.csv` you already have, no new data or setup needed:

- Which machine has the highest average vibration?
- Filter the DataFrame to only the Evening shift rows.
- Add a column flagging rows where `temperature > 76`.

Data Preprocessing with Scikit-Learn

Data Preprocessing for Pima-Indians Diabetes Dataset

(Ref: How To Prepare Your Data For Machine Learning in Python with Scikit-Learn - Jason Brownlee)

Read Data

Import Pima Indians dataset and split into Features (X) and target (Y)

```
import pandas
import numpy

url =
    "https://raw.githubusercontent.com/jbrownlee/Datasets/
names = ['preg', 'plas', 'pres', 'skin', 'test',
        'mass', 'pedi', 'age', 'class']

dataframe = pandas.read_csv(url, names=names)
array = dataframe.values

# separate array into input and output components
X = array[:,0:8]
Y = array[:,8]
```

Rescale Data

- Also called normalization: every attribute is rescaled into the range 0 to 1.
- This helps the optimization algorithms at the core of machine learning, such as gradient descent.
- It also helps algorithms that weight their inputs, such as regression and neural networks.
- It helps algorithms that use distance measures too, such as K-Nearest Neighbors.

Rescale Data

Rescale your data using scikit-learn using the `MinMaxScaler` class.

```
# Rescale data (between 0 and 1)
from sklearn.preprocessing import MinMaxScaler

scaler = MinMaxScaler(feature_range=(0, 1))
rescaledX = scaler.fit_transform(X)

# summarize transformed data
numpy.set_printoptions(precision=3)
print(rescaledX[0:5,:])
```

Rescale Data

After rescaling you can see that all of the values are in the range between 0 and 1.

```
[[ 0.353  0.744  0.59   0.354  0.    0.501  0.234
   0.483]
 [ 0.059  0.427  0.541  0.293  0.    0.396  0.117
   0.167]]
```



```
[ 0.471  0.92   0.525  0.    0.    0.347  0.254
 0.183]
[ 0.059  0.447  0.541  0.232  0.111  0.419  0.038
 0.    ]
[ 0.    0.688  0.328  0.354  0.199  0.642  0.944
 0.2   ]]
```

Standardize Data

- Standardization transforms an attribute so that it has a mean of 0 and a standard deviation of 1.
- It suits attributes that already follow a Gaussian distribution, but with differing means and spreads.
- It suits techniques that assume Gaussian inputs, such as linear regression, logistic regression and linear discriminant analysis.

Intuition

Rescale (MinMaxScaler) and Standardize (StandardScaler) are easy to mix up: rescaling squeezes every value into a fixed [0,1] box regardless of shape, while standardizing centers the data on 0 and measures everything in “number of standard deviations”: which is why standardized values can still go negative or exceed 1, unlike rescaled ones.

Standardize Data

Standardize data using scikit-learn with the StandardScaler class.

```
# Standardize data (0 mean, 1 stdev)
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler().fit(X)
rescaledX = scaler.transform(X)

# summarize transformed data
numpy.set_printoptions(precision=3)
print(rescaledX[0:5,:])
```

Standardize Data

The values for each attribute now have a mean value of 0 and a standard deviation of 1.

```
[[ 0.64   0.848  0.15   0.907 -0.693  0.204  0.468
  1.426]
 [-0.845 -1.123 -0.161  0.531 -0.693 -0.684 -0.365
 -0.191]
 [ 1.234  1.944 -0.264 -1.288 -0.693 -1.103  0.604
 -0.106]
 [-0.845 -0.998 -0.161  0.155  0.123 -0.494 -0.921
 -1.042]]
```

```
[-1.142  0.504 -1.505  0.907  0.766  1.41   5.485
 -0.02  ]]
```

Normalize Data

- Normalizing in scikit-learn refers to rescaling each observation (row) to have a length of 1 (called a unit norm in linear algebra).
- This preprocessing can be useful for sparse datasets (lots of zeros) with attributes of varying scales when using algorithms that weight input values such as neural networks and algorithms that use distance measures such as K-Nearest Neighbors.

Normalize Data

Normalize data in Python with scikit-learn using the Normalizer class.

```
# Normalize data (length of 1)
from sklearn.preprocessing import Normalizer

scaler = Normalizer().fit(X)
normalizedX = scaler.transform(X)
# summarize transformed data
numpy.set_printoptions(precision=3)
print(normalizedX[0:5,:])
```

Normalize Data

Each row is rescaled so that the sum of the squares of its values equals 1: the row's direction is preserved, only its length changes. A simple worked example, for one row of two features. Applied to every row of X, this is exactly what normalizedX above contains: each row now has unit length, so no single value in it can exceed 1 in magnitude.

```
row = [4, 3]                # length =
      sqrt(4^2 + 3^2) = 5
normalized_row = [4/5, 3/5]  # = [0.8, 0.6]
# check: 0.8^2 + 0.6^2 = 1.0
```

Binarize Data (Make Binary)

- You can transform your data using a binary threshold. All values above the threshold are marked 1 and all equal to or below are marked as 0.
- This is called binarizing your data or threshold your data. It can be useful when you have probabilities that you want to make crisp values.
- It is also useful when feature engineering and you want to add new features that indicate something meaningful.

Binarize Data

Create new binary attributes in Python using scikit-learn with the Binarizer class.

```
# binarization
from sklearn.preprocessing import Binarizer

binarizer = Binarizer(threshold=0.0).fit(X)
binaryX = binarizer.transform(X)
# summarize transformed data
numpy.set_printoptions(precision=3)
print(binaryX[0:5,:])
```

Binarize Data

You can see that all values equal or less than 0 are marked 0 and all of those above 0 are marked 1.

```
[[ 1.  1.  1.  1.  0.  1.  1.  1.]
 [ 1.  1.  1.  1.  0.  1.  1.  1.]
 [ 1.  1.  1.  0.  0.  1.  1.  1.]
 [ 1.  1.  1.  1.  1.  1.  1.  1.]
 [ 0.  1.  1.  1.  1.  1.  1.  1.]]
```

Putting It Together: Pipeline

- Scaling the whole dataset before splitting it leaks test information into training.
- A Pipeline chains preprocessing and model into one Estimator.
- It exposes the same fit and predict methods as any single model.
- Inside cross validation, the scaler is refitted on each training fold only.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression

pipe = make_pipeline(StandardScaler(),

    LogisticRegression(max_iter=1000))

# scaler is fitted per fold, never on the held-out data
pipe.fit(X, Y)
predictions = pipe.predict(X)
```

Quick Check: Data Preprocessing

Think About It

You fit a `StandardScaler` on your training data, then need to preprocess the test set before evaluating your model. Should you call `scaler.fit(X_test)` again, or reuse the scaler fitted on the training data?

Quick Check: Data Preprocessing

Answer

Always reuse the scaler fitted on the training data (`scaler.transform(X_test)`, no re-fit). Fitting a new scaler on the test set would compute a *different* mean/std from data the model has never seen, so the same raw value could be scaled differently between train and test, silently corrupting the features the model was trained on. Fitting only ever happens once, on training data; every other split reuses that same fitted scaler. A Pipeline enforces this for you.

Model Evaluation with Scikit-Learn

Classification Model Evaluation for Pima-Indians Diabetis Dataset

(Ref: Metrics To Evaluate Machine Learning Algorithms in Python - Jason Brownlee)

Read Data: the Same Pima Dataset

Same load step as the preprocessing section, repeated so this section stands on its own.

```
import pandas
import numpy

url = "https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-indians-diabetes1.csv"
names = ['preg', 'plas', 'pres', 'skin', 'test', 'mass', 'pedi', 'age', 'class']

dataframe = pandas.read_csv(url, names=names)
array = dataframe.values

# separate array into input and output components
X = array[:,0:8]
Y = array[:,8]
```

Classification Accuracy

- Classification accuracy is the number of correct predictions as a ratio of all predictions made.
- This is the most common evaluation metric for classification problems, and also the most misused.
- It is only suitable when each class has roughly the same number of observations, which is rarely the case.
- It also assumes every kind of prediction error costs the same, which is often untrue.

Classification Accuracy

Accuracy score of approximately 77% accurate

```
from sklearn import model_selection
from sklearn.linear_model import LogisticRegression

seed = 7
kfold = model_selection.KFold(n_splits=10,
                              shuffle=True, random_state=seed)
model = LogisticRegression(max_iter=1000)
scoring = 'accuracy'
results = model_selection.cross_val_score(model,
                                          X, Y, cv=kfold, scoring=scoring)
print("Accuracy: %.3f (%.3f)" % (results.mean(),
                                results.std()))

# Output:
# Accuracy: 0.772 (0.050)
```

Logarithmic Loss

- Logarithmic loss (or logloss) is a performance metric for evaluating the predictions of probabilities of membership to a given class.
- The scalar probability between 0 and 1 can be seen as a measure of confidence for a prediction by an algorithm. Predictions that are correct or incorrect are rewarded or punished proportionally to the confidence of the prediction.

Logarithmic Loss

Smaller logloss is better with 0 representing a perfect logloss. As mentioned above, the measure is inverted to be ascending when using the `cross_val_score()` function.

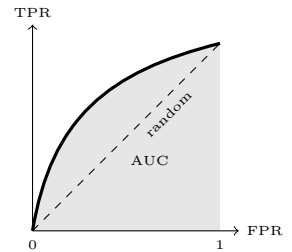
```
seed = 7
kfold = model_selection.KFold(n_splits=10,
                              shuffle=True, random_state=seed)
model = LogisticRegression(max_iter=1000)
scoring = 'neg_log_loss'
results = model_selection.cross_val_score(model,
                                          X, Y, cv=kfold, scoring=scoring)
```

```
print("Logloss: %.3f (%.3f)" % (results.mean(),
                                results.std()))

# Output:
# Logloss: -0.485 (0.057)
```

Area Under ROC Curve

- Area under ROC Curve (AUC) measures how well a binary classifier separates positive cases from negative cases.
- An area of 1.0 is a perfect model, an area of 0.5 is no better than random.
- The curve trades off sensitivity (the true positive rate, also called recall) against specificity (the true negative rate).



The shaded area under the curve is the AUC. The dashed diagonal is a coin-flip model.

Intuition

AUC = 0.5 isn't "50% accurate": it means the model ranks positive and negative cases no better than a coin flip. AUC is threshold-free: it asks how often the model scores a random positive case above a random negative one. That is why it stays meaningful even when the classes are wildly imbalanced, unlike plain accuracy.

Area Under ROC Curve

The AUC is relatively close to 1 and greater than 0.5, suggesting some skill in the predictions.

```
seed = 7
kfold = model_selection.KFold(n_splits=10,
                              shuffle=True, random_state=seed)
model = LogisticRegression(max_iter=1000)
scoring = 'roc_auc'
results = model_selection.cross_val_score(model,
                                          X, Y, cv=kfold, scoring=scoring)
print("AUC: %.3f (%.3f)" % (results.mean(),
                                results.std()))

# Output:
# AUC: 0.829 (0.047)
```

Confusion Matrix

- The confusion matrix presents the accuracy of a model with two or more classes.
- Each cell counts how often an actual class was predicted as a given class.
- Correct predictions land on the diagonal, every mistake lands off it.
- The two mistakes are not the same: a false positive is a false alarm, a false negative is a miss.

	pred 0	pred 1
actual 0	TP	FP
actual 1	FN	TP

The shaded diagonal holds the correct predictions. Everything off it is an error.

Confusion Matrix

The majority of the predictions fall on the diagonal line of the matrix (which are correct predictions).

```
from sklearn.metrics import confusion_matrix

test_size = 0.33
seed = 7
X_train, X_test, Y_train, Y_test =
    model_selection.train_test_split(X, Y,
    test_size=test_size, random_state=seed)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, Y_train)
predicted = model.predict(X_test)
matrix = confusion_matrix(Y_test, predicted)
print(matrix)

# Output:
# [[142  20]
#   [ 34  58]]
```

Classification Report

- Scikit-learn does provide a convenience report when working on classification problems to give you a quick idea of the accuracy of a model using a number of measures.
- The classification_report() function displays the precision, recall, f1-score and support for each class.

Classification Report

You can see good precision and recall for the algorithm.

```
from sklearn.metrics import classification_report

test_size = 0.33
seed = 7
```

```
X_train, X_test, Y_train, Y_test =
    model_selection.train_test_split(X, Y,
    test_size=test_size, random_state=seed)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, Y_train)
predicted = model.predict(X_test)
report = classification_report(Y_test, predicted)
print(report)

# Output:
#               precision    recall  f1-score   support
#
#   0.0         0.81      0.88      0.84         162
#   1.0         0.74      0.63      0.68           92
#
#   accuracy              0.79
#   macro avg              0.78
#   weighted avg           0.78
```

Regression Model Evaluation for Boston House Price Dataset

(Ref: Metrics To Evaluate Machine Learning Algorithms in Python - Jason Brownlee)

Read Data

Import Boston House Price dataset and split into Features (X) and target (Y)

```
import pandas

url =
    "https://raw.githubusercontent.com/jbrownlee/Datasets/master/housing.data"
names = ['CRIM', 'ZN', 'INDUS', 'CHAS', 'NOX',
         'RM', 'AGE', 'DIS', 'RAD', 'TAX', 'PTRATIO',
         'B', 'LSTAT', 'MEDV']
dataframe = pandas.read_csv(url, sep='s+',
                             names=names)
array = dataframe.values
X = array[:,0:13]
Y = array[:,13]
```

Mean Absolute Error

- The Mean Absolute Error (or MAE) is the sum of the absolute differences between predictions and actual values. It gives an idea of how wrong the predictions were.

- The measure gives an idea of the magnitude of the error, but no idea of the direction (e.g. over or under predicting).

Mean Absolute Error

A value of 0 indicates no error or perfect predictions. Like logloss, this metric is inverted by the cross_val_score() function.

```
from sklearn.linear_model import LinearRegression

seed = 7
kfold = model_selection.KFold(n_splits=10,
    shuffle=True, random_state=seed)
model = LinearRegression()
scoring = 'neg_mean_absolute_error'
results = model_selection.cross_val_score(model,
    X, Y, cv=kfold, scoring=scoring)
print("MAE: %.3f (%.3f)" % (results.mean(),
    results.std()))

# Output:
# MAE: -3.387 (0.667)
```

Mean Squared Error

- The Mean Squared Error (or MSE) is much like the mean absolute error in that it provides a gross idea of the magnitude of error.
- Taking the square root of the mean squared error converts the units back to the original units of the output variable and can be meaningful for description and presentation. This is called the Root Mean Squared Error (or RMSE).

Mean Squared Error

This metric too is inverted so that the results are increasing. Remember to take the absolute value before taking the square root if you are interested in calculating the RMSE.

```
seed = 7
kfold = model_selection.KFold(n_splits=10,
    shuffle=True, random_state=seed)
model = LinearRegression()
scoring = 'neg_mean_squared_error'
results = model_selection.cross_val_score(model,
    X, Y, cv=kfold, scoring=scoring)
print("MSE: %.3f (%.3f)" % (results.mean(),
    results.std()))

# Output:
# MSE: -23.747 (11.143)
```

R^2 Metric

- The R^2 (or R Squared) metric provides an indication of the goodness of fit of a set of predictions to the actual values. In statistical literature, this measure is called the coefficient of determination.
- $R^2 = 1$ is a perfect fit, and $R^2 = 0$ means the model does no better than always predicting the mean.
- R^2 is *not* bounded below by 0: a model that fits worse than that mean-only baseline scores negative, which happens easily on an unrepresentative fold.

R^2 Metric

Unlike MAE and MSE above, 'r2' is not negated by `cross_val_score()`: higher is already better, so this mean can be read directly. Note `shuffle=True` in the `KFold` below. This dataset is stored in a sorted order, so without shuffling the folds are unrepresentative, and the score collapses towards the negative- R^2 case from the previous slide.

```
seed = 7
kfold = model_selection.KFold(n_splits=10,
                               shuffle=True, random_state=seed)
model = LinearRegression()
scoring = 'r2'
```

```
results = model_selection.cross_val_score(model,
                                           X, Y, cv=kfold, scoring=scoring)
print("R^2: %.3f (%.3f)" % (results.mean(),
                             results.std()))

# Output:
# R^2: 0.718 (0.099)
```

Quick Check: Model Evaluation

Think About It

The Pima diabetes dataset has roughly twice as many non-diabetic (class 0) cases as diabetic (class 1) cases. A model that always predicts class 0, no matter the input, could reach around 65% classification accuracy. Which of the metrics on the earlier slides would immediately expose this model as useless, and why does plain accuracy fail to?

Quick Check: Model Evaluation

Answer

AUC would immediately expose it: a constant-output model can't rank any example above another, so it scores $AUC \approx 0.5$, no better than random, regardless of class imbalance. The confusion matrix would show the same story directly: an entire empty row or column, since class 1 is never predicted.

Accuracy fails here because it only counts how many predictions matched, and on an imbalanced dataset "always predict the majority class" matches a lot of the time by construction: exactly why accuracy is called out on the first slide of this section as "the most misused" metric.

Quick Check: Regression Metrics

Think About It

Two house-price models are each wrong by a total of 10 lakh rupees across 10 houses. Model A is off by 1 lakh on every house. Model B is perfect on 9 houses and off by 10 lakh on the tenth. They have the same MAE. Which model does MSE prefer, and which would you rather deploy?

Quick Check: Regression Metrics

Answer

MSE strongly prefers Model A. Squaring the errors makes one large mistake cost far more than many small ones: Model A scores $10 \times 1^2/10 = 1$, while Model B scores $100/10 = 10$, ten times worse, even though MAE rates them identically. Which model you deploy depends on the cost of being wrong. If one badly mispriced house is a disaster, use MSE and pick A. If all errors cost the same per rupee, MAE is the honest metric and the two really are equivalent. This is why the two are reported together rather than one replacing the other.